ENGINEERING CHALLENGE

Carrier Dispatch Console

Build a dispatch tool that lets an operations team manage shipments, assign them to vehicles with capacity constraints, and generate optimized multi-stop routes.

Role: Full-Stack Engineer **Time:** 3–4 days **Stack:** Your choice

01 · CONTEXT

What You're Building

Oway is a logistics platform that connects shippers with carriers for LTL (less-than-truckload) freight delivery. When a shipper creates an order, our operations team needs to assign those shipments to available vehicles, each with weight and pallet capacity limits.

Your task is to build a dispatch console — the tool our ops team would use to manage a day's worth of shipments across the LA metro area.

> **We encourage you to use AI tools** (Copilot, Claude, ChatGPT, etc.) to help you build this. We care about the end result and your ability to reason about the decisions — not whether you typed every line yourself. Be prepared to talk through your solution, including any parts AI helped with.

02 · DATA MODEL

Shipments

Each shipment has an origin, a destination, cargo details, and a status. We've provided seed data (see `oway-challenge-seed-data.json`) with 40 shipments and 4 vehicles to get you started.

FIELD	TYPE	NOTES
`id`	string	Unique identifier

FIELD	TYPE	NOTES
`origin`	Address	Pickup location with name, address, city, state, zip, contact, phone, open/close times
`destination`	Address	Delivery location — same shape as origin
`palletCount`	number	Number of pallets
`weightLbs`	number	Total weight in pounds
`description`	string	What's being shipped
`status`	enum	See status progression below
`accessorials`	string[]	Special requirements: `liftgate`, `appointment`, `limited_access`, `hazmat`

Status Progression

Shipments follow a defined lifecycle. Transitions should only move forward — a `PICKED_UP` shipment can't go back to `ASSIGNED`. A shipment can be cancelled from any pre-delivery state.

INITIALIZED → ASSIGNED → PICKED_UP → DELIVERED

Any state → CANCELLED

Vehicles

The seed data includes 4 vehicles with different capacity constraints. A vehicle's assigned shipments cannot exceed its pallet or weight limits.

FIELD	TYPE	NOTES
`id`	string	Unique identifier
`type`	string	`dry_van` or `box_truck`
`maxPallets`	number	Maximum pallet capacity
`maxWeightLbs`	number	Maximum weight capacity

What the Console Should Do

Shipment Management

1. **View all shipments** — with the ability to filter by status, sort by relevant fields, and search. Delivered and cancelled shipments should be visually distinct.
2. **Create a new shipment** — with validation. Origin and destination addresses, pallet count, weight, description, and accessorials.
3. **Progress a shipment's status** — move it forward through the lifecycle. Show the current status clearly and restrict to valid transitions.

Vehicle Assignment

4. **Assign shipments to vehicles** — an ops user selects unassigned shipments and assigns them to a vehicle. The system must enforce capacity constraints (pallets and weight). Show each vehicle's current load and remaining capacity.
5. **View a vehicle's workload** — see all shipments assigned to a vehicle, their total load, and remaining capacity at a glance.

Route Optimization

6. **Generate an optimized route per vehicle** — given a vehicle's assigned shipments, produce an ordered sequence of stops that represents a reasonable route. Each shipment has both a pickup and a delivery — **the pickup must come before the delivery** in the route.
7. **Respect time windows** — each address has `openTime` and `closeTime`. The route should account for these when ordering stops. You don't need to calculate exact drive times, but the route should make a reasonable attempt not to schedule a stop outside its window.
8. **Visualize the route** — show the ordered stops on a map, or as a clear ordered list, or both. The ops user should be able to see the sequence and understand the plan.

> **A note on "optimized":** We're intentionally not defining exactly what optimized means. Minimizing total distance is one approach, but it's not the only one. Think about what an ops team would actually care about, make a choice, and be ready to explain your reasoning.

---

04 · SEED DATA

Working with the Data

The included `oway-challenge-seed-data.json` contains 40 shipments, 4 vehicles, and a depot location (the starting point for all vehicles).

This data is purposely structured with some inconsistencies. How you handle data quality issues is part of the evaluation.

Included in the file

- 40 shipments across the LA metro / Southern California area
- 4 vehicles with different capacities
- A depot address (vehicle starting point)
- Addresses with open/close time windows

Things to keep in mind

- Not all 40 shipments can be assigned — total demand exceeds fleet capacity
- Some shipments have accessorial flags
- The data hasn't been pre-validated
- Geocoding addresses is your responsibility

05 · DELIVERABLES

What to Submit

1. **A working application** (frontend + backend) with the functionality described above.
2. **A README** explaining how to run it, your tech choices, and any assumptions or tradeoffs you made — especially around routing.
3. **Submit via GitHub repo or zip file** so we can review ahead of the follow-up interview.

Use whatever frameworks, databases, and languages you're comfortable with. There's no "right" stack.

> **Be prepared to present your solution** at the follow-up interview. We'll walk through the code together, discuss your architecture and design decisions, and may ask you to extend the solution on the spot.

06 · HOW WE EVALUATE

What We're Looking For

Architecture & Modeling

How you structure the data model, the API layer, and the separation between frontend and backend. Can the system grow?

Problem Solving

How you approach the routing and assignment problem. We don't expect an optimal solution — we want to see you reason about tradeoffs.

Data Handling

How you deal with the seed data — validation, edge cases, and what you choose to do when something doesn't look right.

UI & UX

The console should be usable. Clear status indicators, good information density, and sensible workflows — not just a form and a table.

We're not grading on polish or pixel-perfection. We're looking for evidence of thoughtful engineering: clear data flow, reasonable abstractions, constraint handling, and the ability to explain your choices in the follow-up conversation.